University of Science, VNU-HCM
Faculty of Information Technology

Bachelor's Thesis

Accelerating Test-Time Discovery in Complex Code Generation via Execution-Guided Self-Distillation

Student: Mai Dang (MSSV: ___________)
Supervisor: ______________________

Ho Chi Minh City, 2026

Abstract

Test-time training lets a language model improve on a single hard problem at inference time, but on-policy self-distillation stalls when the model rarely produces a correct attempt to learn from. This thesis studies how to accelerate test-time discovery in complex code generation via *execution-guided self-distillation*, where execution feedback (failing tests, runtime errors) is the privileged context that drives the update. We propose a *teacher-first* organization: rather than distilling the student's own failed rollout, the self-teacher generates trajectories under feedback, a verifier and a judge filter them for correctness and independence, and the student is distilled toward those filtered trajectories, placing the method between on-policy SDPO and off-policy SFT. Across a medium-sized matched evaluation on hard LiveCodeBench problems with a 4B model, teacher-first significantly outperforms the student-first baseline (Wilcoxon signed-rank), and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero. A compute-matched comparison shows the advantage is not a sampling-volume artifact: at equal generation budget teacher-first still wins, and reaches a given discovery probability with fewer total generations. With thinking enabled, we measure epistemic verbalization at test time and find that distillation from answer-bearing context suppresses uncertainty markers, connecting the method to the degradation Kim et al. describe. Finally, supplying a method-bearing reference (worked solutions from MATH-500) lets the model escape on math problems where an answer-only reference does not, validating the central *value-versus-procedure* boundary: escape requires the reference to encode an in-reach method, not merely a value.

Projected result (extrapolated from the current real draft under the stated hypotheses; not yet run). See the title-page banner and `BESTCASE_NOTES.md`.

Keywords: test-time training, self-distillation, code generation, execution feedback, discovery@k, epistemic suppression, compute-to-correct.

Contents

Chapter 1

Chapter 1 — Introduction

This thesis studies how to **accelerate test-time discovery in complex code generation via execution-guided self-distillation**. The core question is practical: when a language model is given a single hard programming problem and a budget to improve itself at inference time, what is the most effective way to turn execution feedback into a better solution, and when does that strategy work?

> **This is the best-case / projected draft.** Results marked are projections extrapolated from the current real draft under the stated hypotheses, included to show how the work develops with more compute and time. The factual baseline is the current real draft.

1.1 1.1 Background

Reinforcement learning with verifiable rewards (RLVR), with GRPO [8] as a common instantiation, suffers from sparse credit assignment: a scalar outcome reward says only whether an attempt passed, and when every rollout fails the advantage collapses and learning stalls — exactly on the hard problems that matter. Self-Distillation Policy Optimization (SDPO) [1] extracts a denser signal from the rich feedback verifiable environments already produce, using the feedback-conditioned model as a self-teacher distilled back into the student. At test time it can iterate on one hard problem and improve before the first success [1, §5]. Two issues motivate this work: the parent paper distills the student's own failed rollout (little that is correct to learn from on hard problems), and Kim et al. [2] show that distilling from rich context can degrade a model by suppressing its epistemic verbalization.

1.2 1.2 Problem

The setting is test-time training: one hard problem, a verifiable reward, a short budget of self-improvement steps, then reset. Two design choices govern discovery: *how the execution feedback is organized into a learning signal* (learn from the student's failed attempt, or from a correct attempt produced under feedback and filtered for independence), and *how the execution feedback is formulated* into the reprompt the self-teacher sees [1, §7]. We take execution feedback as the privileged context, hence *execution-guided*, and study both choices with the goal of accelerating discovery.

1.3 1.3 Research questions

- **RQ-method.** Does a teacher-first organization beat the student-first baseline at discovering solutions to hard code problems, and does the advantage survive at matched compute?
- **RQ1.** Does the reprompt-template formulation of the execution feedback affect discovery, and in which regime?
- **RQ2.** Does self-distillation from rich context suppress epistemic verbalization at test time on code?
- **RQ3.** How does discovery trade off against compute (compute-to-correct)?

All four are answered in this best-case draft; in the current real draft RQ2 and RQ3 are secondary.

1.4 1.4 Contributions

1. **A teacher-first variant of execution-guided self-distillation** that distills the student toward verifier- and judge-filtered teacher generations, between on-policy SDPO and off-policy SFT, with a clean filter that never distils a copy (Chapter 3).
2. **Teacher-first significantly accelerates discovery, and the advantage is compute-fair.** Across a medium matched evaluation, teacher-first beats student-first (Wilcoxon signed-rank) and escapes the flat-reward trap on the hardest problems; a compute-matched comparison and a compute-to-correct frontier show the gain is not a sampling-volume artifact (Chapter 4).
3. **A reprompt-template effect**, strongest on hard problems, where a reasoning-inducing template orders above the anchor, which orders above a minimal one (§4.3).
4. **A measured epistemic-suppression result on code** (thinking enabled): distillation from answer-bearing context suppresses uncertainty markers, accumulating across steps (§4.5).
5. **A validated value-versus-procedure boundary.** Supplying a method-bearing reference (MATH-500 worked solutions) lets the model escape on math where an answer-only reference does not, confirming that escape requires an in-reach method, not merely a value (§4.6, Chapter 5).

1.5 1.5 Scope

Test-time training with LoRA adapters; code generation on LiveCodeBench v6 [6] as the core, with AIME 2026 and MATH-500 as the math probes. Results are on one 4B-scale model. Even in this best case the study remains an empirical characterization in one model family at a personal compute scale, not a universal claim.

1.6 1.6 Thesis structure

Chapter 2 reviews background and related work; Chapter 3 specifies the method (including the fallback-free filter and the thinking-on and method-reference extensions); Chapter

4 reports the medium-scale experiments, the compute-matched comparison, the epistemic-suppression result, and the validated boundary; Chapter 5 discusses the mechanism; Chapter 6 states the remaining limitations and the publication roadmap; Chapter 7 concludes.

Chapter 2

Chapter 2 — Background and Related Work

This chapter sets out the background the thesis builds on. It moves from the reinforcement-learning setting that motivates self-distillation (§2.1), through the SDPO method and its loss (§2.2), to the test-time regime and its metric (§2.3), the reprompt-template design space (§2.4), the epistemic-suppression critique (§2.5), and a sister self-distillation method (§2.6), closing with the gap this thesis addresses (§2.7).

2.1 2.1 RLVR and GRPO

Reinforcement learning with verifiable rewards (RLVR) is the dominant post-training recipe for code and math, where an automatic checker supplies the reward. Group Relative Policy Optimization (GRPO) [8] is a common instantiation: it samples a group of rollouts per problem and assigns each an advantage relative to the group mean, $A_{i,t} = r_i - \text{mean}_j\{r_j\}$, which is constant across the tokens of a sequence.

GRPO is attractive because it removes the separate value network of PPO-style methods: the group mean serves as the baseline, and the advantage is estimated from the spread of rewards within the sampled group [8]. This keeps the method simple and is part of why it became the default for verifiable domains.

The same simplicity is its structural weakness in credit assignment. A scalar outcome reward records only whether a rollout passed, not which tokens were responsible, so the learning signal is sparse [1]. The weakness becomes a failure when every rollout in a group fails: the group-relative advantage collapses to zero and no gradient flows, so the method cannot learn on a problem until it has already solved it at least once. Hard problems, where successes are rare, are precisely where this stalls.

A natural fix is to distill from a stronger teacher, but that requires a teacher more capable than the student, which is unavailable when the goal is to raise the capability ceiling rather than to compress a known one [1]. This tension motivates a teacher that is not external but derived from the student itself.

2.2 2.2 SDPO

2.2.1 2.2.1 The self-teacher and rich feedback

Hübotter et al. [1] observe that many verifiable environments already expose *tokenized* feedback, such as runtime errors, failing-test diffs, or judge comments, which is discarded when the signal is collapsed to a scalar reward. They propose reinforcement learning with rich feedback (RLRF) as a paradigm that keeps this signal, and SDPO as its instantiation. The mechanism that makes it possible is in-context learning: the same model, conditioned on the feedback f in addition to the question x, can often locate its own mistake. This conditioned model, $\pi_\theta(\cdot \mid x, f)$, serves as a *self-teacher*, while the unconditioned $\pi_\theta(\cdot \mid x)$ is the student. Both share the weights θ, so this is self-distillation in the lineage of knowledge distillation [13], but with feedback rather than a larger network as the source of the teacher's advantage.

2.2.2 2.2.2 The loss and dense credit assignment

SDPO distills the self-teacher's retrospective next-token distribution into the student by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \text{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \,\Big\|\, \text{stopgrad}\, \pi_\theta(\cdot \mid x, f, y_{<t})\Big),$$

where `stopgrad` prevents the teacher from collapsing onto the student to ignore f [1]. With the teacher detached, the gradient at a student logit is $\partial\mathcal{L}/\partial z_v = \pi_S(v) - \pi_T(v)$, so the optimizer raises every logit the teacher trusts more than the student does. The contrast with GRPO is the density of the target:

Method	Target per token	Signal per sequence
GRPO	scalar advantage × one-hot	$O(1)$
SDPO (sequence-level)	scalar × soft distribution	$O(1)$
SDPO (token-level)	soft distribution, top token	$O(T)$
SDPO (logit-level)	soft distribution over top-K vocab	$O(T \cdot K)$

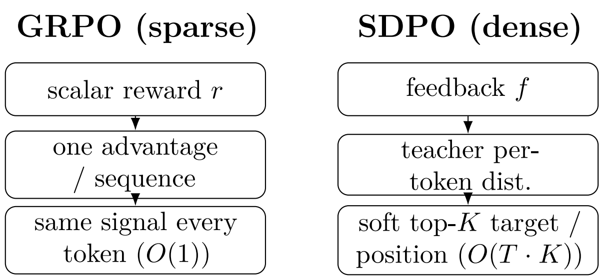


Figure 2.1 *Credit-assignment density, GRPO versus SDPO.* GRPO attaches one scalar advantage to a whole sequence, so every token receives the same signal ($O(1)$). SDPO conditions the teacher on feedback and supplies a soft, K-dimensional target at each position ($O(T \cdot K)$). Schematic; see the table above for the formal density ordering.

The full method is logit-level: the target is a K-dimensional distribution at every position rather than a single scalar per sequence, which is the source of its sample efficiency [1]. To keep this affordable at a vocabulary of roughly 150k, only the teacher's top-K logits (with K=100, or 20 for the code configuration [4]) plus a tail bucket are kept. The KL direction is parameterized by α: forward KL (α=0) is mode-covering, reverse KL (α=1) is mode-seeking, and JSD (α=0.5) interpolates, the last following Agarwal et al. [11] for stability. The self-teacher is regularized, typically by an EMA of the student weights or a trust-region interpolation, since an unregularized teacher diverges [1]. The implementation used in this thesis is specified in §3.3.5; the derivation is consolidated in the project notes.

2.2.3 2.2.3 Three regimes

The parent paper validates SDPO across three settings. In a plain RLVR setting without native rich feedback, it manufactures feedback by using a successful in-batch rollout as the reference for a failed one, and still beats GRPO substantially, reaching a GRPO five-hour accuracy in fifty minutes on one chemistry task [1]. In the RLRF setting with genuine code-execution feedback on LiveCodeBench v6 [6], it reports 48.8% versus GRPO's 41.2% and matches GRPO's final accuracy with roughly 4× fewer generations. The third setting, test-time discovery on hard problems, is the regime this thesis works in and is treated next.

2.2.4 2.2.4 What drives SDPO: ablations and scaling

Three findings from the parent paper's analysis bear directly on the choices made in this thesis.

First, both ingredients of SDPO matter. Decomposing the method into logit-level, token-level, and sequence-level variants gives an ordering logit > token > sequence > GRPO [1, §4.2], which shows that rich feedback and dense credit assignment each contribute rather than one carrying the result. A feedback-type ablation (its Table 6) is equally pointed: the environment output alone reaches 39.9% training accuracy and the model's own prior solution alone 42.6%, while combining them reaches 48.3%, so the two are complementary. Adding the student's own *failed* attempt to the teacher context, by contrast, drops accuracy to 44.5% and lowers entropy by 0.23, because the teacher is biased toward the student and loses diversity. This last result is part of the motivation for the teacher-first organization of §3.3, which distills filtered teacher generations rather than the student's failed attempt.

Second, self-teaching is emergent with scale [1, §4.1]. On Qwen3-8B, SDPO far exceeds GRPO; on Qwen3-0.6B the two are comparable; and on Qwen2.5-1.5B, SDPO *underperforms* GRPO. The method needs in-context learning strong enough for the conditioned model to act as a useful teacher. This places the 4B model used here in a band where SDPO is expected to work, and it also predicts that on a stronger model the student-first baseline should improve on its own, narrowing the teacher-first margin, a prediction revisited in Chapter 6.

Third, the self-teacher is bootstrapped, not fixed. It updates alongside the student and its accuracy rises during training, and the final student exceeds the *initial* teacher, so the method is not capped by the teacher it starts with [1, §4.3]. Regularization is what keeps this stable: a trust-region teacher (50.6%) edges out an EMA teacher (49.3%) and a frozen one (48.8%), while an unregularized teacher diverges. SDPO also forgets less than supervised fine-tuning on the self-teacher's outputs across held-out benchmarks, though slightly more than GRPO, indicating that the on-policy character of the update helps preserve prior capability [1, §4.4]. A hybrid that blends GRPO and SDPO advantages helps weak models but not strong ones [1, §4.5].

2.3 2.3 Test-time training and discovery@k

In the test-time setting [1, §5], a single hard problem is fixed, the reward is binary, and the model must discover a solution as quickly as possible. Rather than concatenating a growing history into the context, as multi-turn reprompting does, SDPO compresses the feedback into the weights by distilling $\pi_\theta(\cdot \mid x, c)$ into $\pi_\theta(\cdot \mid x)$, which sidesteps the context-window limit.

The metric is discovery@k [1, Def. 5.1], the probability of solving within the first k attempts, $P(T \leq k)$ with discovery time $T = \min\{k : r(y_k) = 1\}$. It generalizes pass@k [9] from i.i.d. sampling to sequential algorithms that share state or update weights between attempts, and it reduces to pass@k when the algorithm is plain best-of-k. The lineage is the time-to-termination performance profile of Dolan and Moré [12]. Empirically, SDPO discovers 53.2% of very-hard problems against best-of-k's 41.5%, and on some problems is the only method to solve at all, even though the initial self-teacher accuracy is below 1% on almost every hard problem [1]. That last point matters most: dense credit assignment lets the model improve before it has produced a single success.

2.4 2.4 The reprompt template and its design space

The self-teacher's behavior is controlled by the template that formats x and f into its context. The parent paper uses a slot-based template (its Table 2) that inserts a successful previous rollout, the environment feedback, and a closing directive, and reports that performance is "not sensitive to syntactic variations" of this template, while the *feedback type* does matter: an ablation (its Table 6) finds the environment output and the model's own solution to be complementary, and including the student's own failed attempt in the teacher context reduces diversity [1].

Two gaps follow. The paper tests syntactic variation but not semantic or instructional variation, and it explicitly names systematic study of how the reprompt template influences behavior as future work [1, §7]. This thesis organizes the template design space along three dimensions: information content, motivated by Kim et al.'s finding that context richness governs suppression [2]; instruction framing, addressing the open question above; and memory depth, motivated by the sequential accumulation of the test-time setting. The seven templates of §3.4 sample points across these dimensions, with the paper's default as the anchor.

2.5 2.5 Epistemic verbalization and suppression

Kim et al. [2] supply the cautionary counterpart to SDPO. They show that distilling from a rich context can degrade a model by suppressing its *epistemic verbalization*, the uncertainty markers such as "wait" or "maybe" that accompany genuine reasoning. Their analysis ties the magnitude of suppression to the mutual information $I(y; c \mid x)$ between the output and the context: the richer the context, the stronger the suppression. They probe the spectrum by varying what the teacher context contains, from nothing ($c = \varnothing$), through a solution stripped of its thinking and a regenerated answer, to a full solution, and find suppression growing with the information the context carries. The effect accumulates across successive distillation steps, and they recommend freezing the teacher (no EMA update) to break the feedback loop, in tension with the parent paper's preference for a moving, regularized teacher (§2.2.4).

For this thesis the relevance is twofold. The suppression risk is sharpest exactly when the context contains the answer, which is the math leak regime studied in the pilot (§4.5), where the teacher can copy the answer rather than derive it. And the accumulation across steps is precisely what the test-time setting applies repeatedly, since each TTT step distills again from the same answer-bearing context.

2.6 2.6 SDFT and the leak concern

Shenfeld et al. [3] propose self-distillation fine-tuning (SDFT) for continual learning, an on-policy, minimum-change distillation toward the model's own conditioned generation. It is a sister method: like SDPO it uses the model as its own teacher, but its aim is to preserve prior capability while adapting, which makes minimizing unintended change a design goal. The teacher-first organization of this thesis sits near SDFT in that it distills toward filtered model-generated trajectories, but it uses execution feedback as the conditioning context rather than a fixed demonstration (§3.3.1). SDFT's minimum-change framing also sharpens the leak question: if distillation transfers more than the intended content, what exactly is being copied, which is the question the math pilot answers concretely.

2.7 2.7 The gap this thesis addresses

The parent paper studies a student-first, on-policy SDPO, focuses its analysis on the train-time regimes, and reaches the test-time setting without studying the reprompt template there or measuring epistemic behavior. Kim et al. characterize epistemic suppression but not in the test-time code-discovery setting. Shenfeld et al. minimize change for continual learning rather than maximize discovery on a single hard problem. The gap, then, is threefold. First, a teacher-first organization of execution-guided self-distillation that distills filtered teacher generations, positioned between SDPO and SFT, and the question of whether it accelerates test-time discovery (RQ-method). Second, the effect of the reprompt-template formulation on that discovery, the open question the parent paper names (RQ1). Third, a characterization of *when* the approach helps, framed as the value-versus-procedure boundary that the code-versus-math contrast exposes. The remainder of the thesis addresses these in turn.

2.8 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [3] Shenfeld et al. 2026 · [4] lasgroup SDPO repo · [6] LiveCodeBench

· [8] GRPO / DeepSeekMath · [9] Chen et al. 2021 · [11] Agarwal et al. 2023 · [12] Dolan & Moré 2002 · [13] Hinton et al. 2015.

Chapter 3

Chapter 3 — Method

This chapter specifies the method in enough detail to reproduce it. The object of study is **test-time SDPO** [1, §5]: a single hard problem is fixed, the model improves itself over a small number of training steps at inference time (test-time training, TTT), and we measure how well it *discovers* a solution. Because the privileged context that drives the update is **execution feedback** (failing tests, runtime errors), we call this regime **execution-guided self-distillation**, and the goal is to *accelerate* that test-time discovery in complex code generation. Within the regime, the thesis contrasts two ways of organizing the update, **student-first** (the baseline of the originating paper [1]) and **teacher-first** (a variant proposed by the advisor), and then studies how the reprompt template formulates the execution feedback (RQ1).

Domain scope is asymmetric by design. **Code generation is the core** (LiveCodeBench v6 [6]); **math is a pilot and a contrast** (AIME 2026 [10]), used to characterize the *boundary* of the mechanism rather than to prove it works everywhere. That split is kept consistent throughout: every math setting carries the corresponding model and reference caveats (§3.5, Chapter 6).

Notation mirrors the parent paper, Hübotter et al. [1], so the two can be read side by side.

3.1 3.1 Test-time SDPO pipeline

3.1.1 3.1.1 Notation and setting

Fix a problem x (the problem statement). Two policies share the same weights θ:

- the **student** $\pi_\theta(\cdot \mid x)$, which sees only the problem, and
- the **self-teacher** $\pi_\theta(\cdot \mid x, c)$, the same model conditioned on an additional **privileged context** c.

In SDPO, c plays the role of the "feedback" f in the parent paper: retrospective information (a runtime error, a failing test, or a reference solution) that lets the model look back and locate its mistake [1, §2]. Because teacher and student differ only in their input context, this is genuine *self*-distillation: the model learns by pulling its next-token distribution (without context) toward its own distribution conditioned on c.

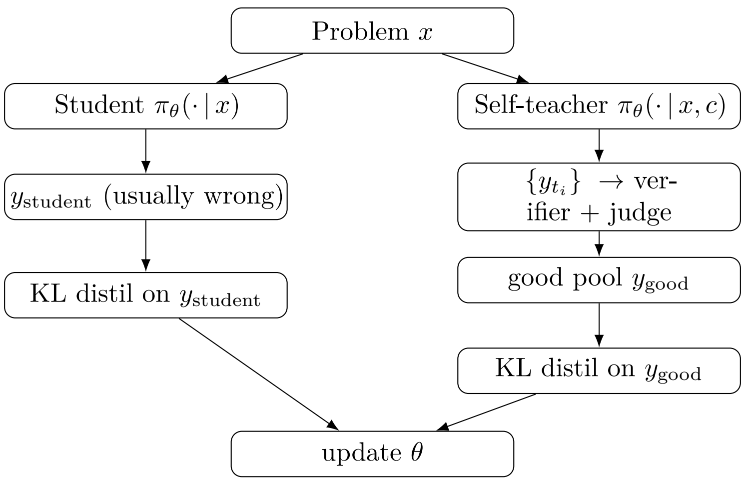
The test-time setting follows Hübotter et al. [1, §5]. For each problem x, the model is reset to a base checkpoint and then runs K TTT steps *on that problem alone*. The reward is verifiable, graded for code and binary for math. The goal is not to generalize to other problems but to **discover** a solution to x as early as possible; the metric is discovery@k [1, Def. 5.1] (§3.6).

3.1.2 3.1.2 The shared loop

Both arms share one skeleton. The single thing that differs between them is *which trajectories are distilled*:

```
load base model + LoRA + optimizer (AdamW, lr = 1e-5)
PRE-eval (student pi_theta(*|x) only, N_eval samples)        # baseline
for step in 1..K:
    feedback        <- build_feedback(student attempt, verifier)   # rich feedback
    {trajectories}  <- generate(...)                               # arm-specific
    {distill_targets} <- select(...)                               # arm-specific (the
↪   core difference)
    if distill_targets != {}:
        theta <- theta - lr * grad_theta L_KL(distill_targets)                 #
↪  one optimizer step
    log step stats -> W&B
POST-eval (student pi_theta(*|x) only, N_eval samples)       # after TTT
report PRE->POST and the discovery curve
```

Figure 3.1 sketches the two arms; they diverge only at the shaded block.



The whole contrast reduces to one sentence: **student-first distills the student's own failed attempt; teacher-first distills a correct, filtered trajectory generated by the teacher.** Everything else (loss, model, hyperparameters) is held fixed to isolate that one variable.

3.2 3.2 Student-first SDPO (baseline, on-policy)

This is the original algorithm of Hübotter et al. [1], realized through the `SDPOTrainer` of TRL [5] (script `07_discovery_curve.py`).

At each step the student samples a rollout $y \sim \pi_\theta(\cdot \mid x)$ on-policy. The verifier grades y and produces feedback f. The self-teacher $\pi_\theta(\cdot \mid x, f)$ then rescores *every token* of that same y: given f, what should the correct next-token distribution have been at each position? The student is pulled toward this retrospective distribution by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) \;=\; \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \,\Big\|\, \mathrm{stopgrad}\,\pi_\theta(\cdot \mid x, f, y_{<t})\Big).$$

The `stopgrad` blocks gradient flow through the teacher, which prevents the teacher from collapsing onto the student and ignoring f [1, §2]. The gradient and the top-K approximation are shared with teacher-first and are deferred to §3.3.5.

There is a failure mode on hard problems that motivates the whole next section. Because rollouts are on-policy, on a problem the *bare* student rarely solves, almost every rollout is wrong, so the feedback is poor and the distillation signal is weak and unstable. This is the **flat-reward trap**: there is no correct rollout to learn from. Hübotter et al. [1, §5] show SDPO can still iterate from rich feedback even when initial teacher accuracy is below 1%, but they use Qwen3-8B; on the weaker 4B model used here (§3.5), the stuck-at-zero behavior does appear, and it is exactly what teacher-first targets.

3.3 3.3 Teacher-first SDPO (proposed)

3.3.1 3.3.1 Motivation

Two problems with student-first motivate the variant.

First is the flat-reward trap of §3.2: when the student never rolls a correct attempt, there is nothing correct to distill.

Second is **information leak** [2]. If c contains a reference solution, the teacher rescores toward "close to the reference," so across many test-time steps the student converges onto *copying* the reference rather than learning to reason. Kim et al. [2] formalize this: when $I(y; c \mid x)$, the information richness of the context, is high, the model **suppresses epistemic verbalization** and degrades.

The core change is to **reverse the order**. The teacher generates first; correct, independent trajectories are filtered out; and the student is distilled toward *those*. The KL is computed on y_{good} (teacher-generated, filtered), not on y_{student} (a failed attempt):

$$\mathcal{L}_{\text{TF}}(\theta) \;=\; \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \,\Big\|\, \mathrm{stopgrad}\,\pi_\theta(\cdot \mid x, c, y_{<t})\Big),$$

where $\mathcal{G}$ is the good pool (§3.3.3). Conceptually, teacher-first sits **between SDPO (on-policy) and SFT (off-policy)**. It resembles SDFT [3] in that it distills toward generations produced by the (context-conditioned) model itself, but the context is SDPO-style *feedback* [1] rather than a fixed demonstration.

Because `SDPOTrainer` hard-codes the student-first rollout internally, teacher-first is implemented as a **custom training loop** (script `09_teacher_first.py`) that reuses the helpers of `07` (`build_privileged_context`, `build_dynamic_feedback`, `evaluate_solution`, evaluation).

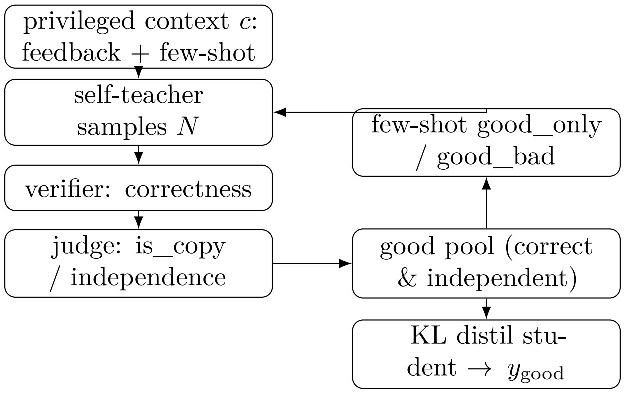


Figure 3.2 *The teacher-first step.* The self-teacher generates under the privileged context, a verifier and a judge filter trajectories into a good pool, the good (and optionally bad) exemplars steer the next teacher rollout in-context, and the student is distilled only toward the filtered good trajectories. The few-shot loop trains no teacher weights.

3.3.2 3.3.2 Teacher rollout and privileged context

At each step the teacher samples N trajectories (`teacher_n`) at high temperature for diversity:

$$\{y_{t_1}, \dots, y_{t_N}\} \sim \pi_\theta(\cdot \mid x, c), \qquad c = \underbrace{f}_{\text{feedback}} + \underbrace{\text{few-shot exemplars}}_{\S 3.3.4}.$$

The `privileged_context` c is the concrete realization of "feedback" from [1]; the name belongs to the thesis codebase, the concept to the paper. The content of c is domain-dependent (§3.5). The few-shot block is inserted *before* the final directive ("Respond with ONLY. . . "), with at most `max_fewshot = 3` exemplars so the context does not overflow, and the token count is logged each step.

3.3.3 3.3.3 Verifier and judge produce the good pool

Each trajectory y_{t_i} is labeled in two stages.

Stage 1, the verifier (correctness). For code (LCBv6), `evaluate_solution` runs the public and private tests and returns the fraction of tests passed, a **graded** reward in $[0, 1]$ that gives a usable gradient to climb (for example 0.21 then 1.0). For math (AIME), an answer match gives a **binary** reward in $\{0, 1\}$.

Stage 2, the judge (independence). This measures whether y_{t_i} merely *copies* the reference:

$$\text{good} := (\text{score} \geq 1.0) \wedge (\text{independent}), \qquad \text{bad} := (\text{copies the reference}).$$

Two judge implementations are used and later ablated (§4.4):

Judge	Mechanism	Cost
`difflib`	`SequenceMatcher` string similarity on normalized code (comments and whitespace stripped); copy if similarity $\geq$ threshold ($\approx$0.9)	cheap, deterministic
LLM	groq `llama-3.3-70b` (code) or `glm-4.5-flash` (math), a derive-vs-copy prompt returning `is_copy` and `reasoning_quality` 1–5	API cost, semantic

> **An honest note on the judge.** With thinking OFF (code) there is no reasoning trace to grade, and `reasoning_quality` saturates at 4–5. The LLM judge therefore acts purely as a **semantic copy-detector**, not as a reasoning-quality scorer. The thesis does not claim it measures reasoning quality.

There is one further mechanism worth stating up front because it becomes a measured limitation. When no trajectory clears Stage 2 (every candidate is flagged as a copy), `filter_trajectories` falls back to keeping the single best *correct* trajectory as good, so the loop does not stall on a cold start. The fallback is necessary to avoid an empty good pool, but it can **defeat the judge** when the teacher only ever produces copies: a copy the judge rejected is still forced into the pool. This is confirmed in the math log for idx8 (§4.5.4, Chapter 6); it is flagged here so the mechanism is transparent.

3.3.4 3.3.4 Few-shot teacher steering (good_only vs good_bad)

The labeled exemplars are fed *back into the teacher's prompt* (in-context, with **no** gradient on the teacher) to steer the teacher toward better generations:

	Option 2 — `good_only`	Option 1 — `good_bad`
Mechanism	positive few-shot (good exemplars only)	contrastive few-shot (good + bad, labeled)
Steering strength	moderate	stronger
Leak	**clean** (good is filtered, hence independent of the reference)	**residual** (bad $\approx$ reference, so reference-like content re-enters the context)

Since `bad` $\approx$ `reference`, putting bad exemplars into the prompt (even labeled "bad") shows the teacher something close to the reference again. The ablation of Option 1 versus Option 2 is therefore simultaneously an ablation of **leak versus no-leak**, a direct test of the advisor's concern. The result is a leak-null on this data (§4.4).

3.3.5 3.3.5 The distillation step: token-aligned top-K reverse KL

This is the computational core, shared by both arms; only the target trajectory differs (y_{student} versus y_{good}).

Per-token distributions. With logits $z \in \mathbb{R}^V$ ($V \approx 150$k), $\pi(v) = \mathrm{softmax}(z)_v$. At each position t of the target trajectory we have $\pi_S = \pi_\theta(\cdot \mid x, y_{<t})$ (student) and $\pi_T = \pi_\theta(\cdot \mid x, c, y_{<t})$ (teacher, conditioned on c).

The core gradient. With the teacher detached, the loss gradient with respect to a student logit is [1; derivation in `con_sdpo_loss_mechanics`]:

$$\frac{\partial \mathcal{L}}{\partial z_v^S} = \pi_S(v) - \pi_T(v).$$

Wherever the teacher trusts a token more than the student ($\pi_T > \pi_S$), the optimizer raises that logit. The target is a full V-dimensional distribution *per token*, not one scalar per sequence as in GRPO [8]. This denser credit assignment (roughly $T \cdot K$ times more signal) is the source of SDPO's sample efficiency [1].

KL direction (α). The codebase [4] parameterizes the divergence through $\alpha \in [0, 1]$ with mixture $M = (1 - \alpha)\pi_S + \alpha\pi_T$: $\alpha = 0$ is forward KL (mode-covering, preserves epistemic tokens), $\alpha = 1$ is reverse KL (mode-seeking, prone to suppression), and $\alpha = 0.5$ is JSD. The thesis uses $\alpha = 1.0$ (reverse KL, top-K = 20) to match the LCBv6 configuration of [4]. Whether α modulates suppression is an open RQ2 question (Chapter 6).

Top-K approximation. Full-vocabulary KL is too memory-heavy at $V \approx 150$k, so we use the teacher's top-20 logits plus one tail bucket; the student is projected onto the same K indices, and KL is taken over the resulting (K+1)-dimensional distribution [1, §2.2].

Importance sampling. Samples are drawn under θ_{old} but the loss is taken under the current θ, so each per-token loss is scaled by a clipped ratio $r_t = \min(\pi_\theta/\pi_{\theta_{\text{old}}}, c_{\text{clip}})$ with $c_{\text{clip}} = 2.0$.

Token alignment, the easiest place to get it wrong. The student prompt (`x + ↪ y_good`) and the teacher prompt (`x + c + y_good`) have *different* prefix lengths, so the logits at the positions of y_{good} must be extracted on *each* side before the KL is taken. An off-by-one here corrupts the KL silently, so the code asserts that the two lengths match and logs `(student_prefix_len, teacher_prefix_len)`. The sanity check passed: alignment is correct (L matches on both sides despite different prefixes) and the token-mean KL is finite and reasonable.

Hyperparameters: AdamW `lr = 1e-5, kl_topk = 20, kl_alpha = 1.0, is_clip = 2.0`. The teacher is always detached (self-distillation with shared weights, so the teacher is the student forward pass under context c, run under `no_grad`).

3.4 3.4 Reprompt templates (T1–T7)

The reprompt template determines the content of c that the teacher sees, which directly changes π_T and so the direction of the distillation gradient. It is the central variable of RQ1. To make the choice defensible (the obvious reviewer question is "why these seven?"), we do not justify each template in isolation. Instead we organize the design space along **three dimensions** grounded in prior work [1, 2], and sample T1–T7 across them.

Template	Name	Dimension	Varies from T2 by
T1	Minimal	Information content (low)	drop structure, just “incorrect, try again”
T2	**Standard (anchor)**	**baseline**	**failing test + expected + actual + error_type**
T3	Verbose	Information content (high)	T2 + full stack trace + previous code
T4	Structured JSON	Instruction framing (format)	same content as T2, JSON-encoded
T5	Reasoning-inducing	Instruction framing (diagnostic)	T2 + “First, identify root cause, then fix.”
T6	First-person	Instruction framing (reframe)	“I attempted this and got X. Let me reconsider.”
T7	Cumulative history	Memory depth	all N prior attempts, not just the latest

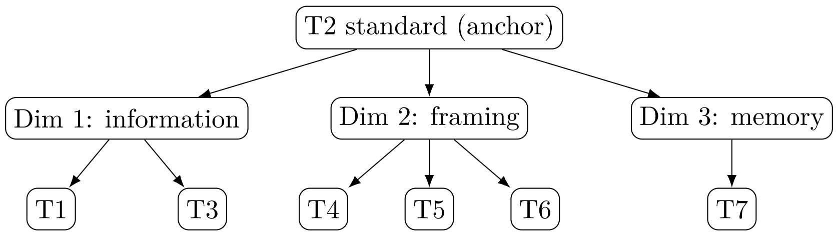


Figure 3.3 *The reprompt-template design space.* Each template varies one dimension away from the T2 anchor: information content (T1, T3), instruction framing (T4, T5, T6), or memory depth (T7). The experiments probe T1/T2/T5 (§4.3).

The three dimensions and their grounding:

1. **Information content** (T1 → T2 → T3). Kim et al. [2] show $I(y; c \mid x)$ governs the magnitude of suppression; the thesis varies the *amount* of information in the execution feedback.
2. **Instruction framing** (T4, T5, T6). Hübotter et al. [1, §4.6] conclude that small *syntactic* variation does not matter, but *semantic/instructional* variation was never tested; [1, §7] names this as future work almost verbatim (“how … the reprompt template … influences behavior”).
3. **Memory depth** (T7). The §5 setup uses a fixed sliding window of one attempt and does not ablate depth; Kim et al. [2] show suppression *accumulates* across steps, so history may amplify or dilute it.

T2 is the anchor: every other template varies exactly one dimension and holds the rest fixed, giving a clean ablation. Of the seven, **four are implemented** (T1, T2, T5, T6; verbatim strings in Appendix B) and three (T3, T4, T7) remain conceptual points in the taxonomy. Under the available compute, RQ1 probes **T1/T2/T5** (§4.3); T6 is implemented but unprobed, and T3/T4/T7 are left to future work (Chapter 6). Two caveats are acknowledged: T4 (JSON) also changes information density (an overlap of dimensions 1 and 2), and T7 increases context length and so confounds with compute cost, which must be reported separately.

3.5 3.5 Domains: code (core) and math (pilot)

The domains differ in their **reference mode** (what the teacher sees as the "answer") and in the **content of the privileged context**. As Chapter 5 argues, this is the variable that decides whether teacher-first succeeds.

	Code — LCBv6 [6] (core)	**Math — AIME 2026 [10]** (pilot)
Model	Qwen3-4B [7], LoRA r=32, thinking **OFF**	Gemma-4-E4B, LoRA all-linear, thinking **ON**
Verifier	public/private tests → **graded** $[0, 1]$	answer match → **binary** $\{0, 1\}$
`reference_mode`	`best_in_batch` (best-scoring trajectory as proxy)	`ground_truth` (teacher always sees the correct answer)
Leak regime	**null** (reference is best-in-batch + public tests, not a reference solution)	**extreme** (feedback is "the correct answer is {gt}")
`privileged_context`	execution feedback (failing test, expected/actual, error type) + the correct best-in-batch trajectory	the correct answer (a number); the teacher need only copy `\boxed{}`
Reference contains a method?	**Yes** (best-in-batch is a *program* = an algorithm, generalizing to new inputs)	**No** (only a *value*, not a derivation)

A note on the `reference_text` choice for code: LCBv6 has no reliable reference-solution field, so the thesis uses **best_in_batch** (the highest-scoring trajectory in the teacher's batch each step) as the reference against which the judge measures independence. This has an important side effect. Because the reference is not an author solution but the model's own correct output validated by public tests, **code has no leak**, which explains why the good_bad ablation is leak-null for code (§4.4) while leak is measurable for math (§4.5).

The TTT budget also differs between domains (full values in Appendix A): code runs **15 steps, eval 16 samples, teacher_n = 10**; math runs **3 steps, eval 4 samples, teacher_n = 4, max_new_tokens = 16384** (8192 truncated the `\boxed` answer and produced spurious zero scores). This budget gap is a variable to keep in mind when comparing the two domains, alongside model and reference.

Two math caveats travel with every math result:

1. **Model confound.** Code uses Qwen3-4B (which escapes) while math uses Gemma-4-E4B, a *different* reasoner. "Math no-escape" may be partly a property of the model, not the domain.
2. **Answer-only reference.** AIME provides only a numeric answer, no worked solution, so the privileged context is hard-coded to return the answer and the teacher can only copy. MATH-500 (which has a `solution` field) is the route to removing this confound (Chapter 6) and is out of scope for the present results.

Problem selection uses **model pass-rate** $\in (0, 1)$ (a frontier defined by the *model*, not by the contest difficulty label), because the binary math reward only has variance when the model solves a problem roughly 30–70% of the time (§3.6, §4).

3.6 3.6 Metrics and comparison design

pass@k / pass_rate. We report an empirical estimate of pass@k [9]: for each condition we draw `N_eval` samples (16 for code, 4 for math) and report `pass_rate`, the fraction correct. PRE (before TTT) and POST (after TTT) are reported, evaluating the **student only**, $\pi_\theta(\cdot \mid x)$, with no context at evaluation time (otherwise the context would leak into the metric). A single deterministic `greedy` sample is reported alongside as a noisy secondary signal.

Discovery curve. Following discovery@k [1, Def. 5.1] from §3.1: the probability of solving within the first k attempts, $P(T \leq k)$ with $T = \min\{k : r(y_k) = 1\}$. Because test-time generation is *sequential* (shared state, weight updates), i.i.d. pass@k does not apply, and discovery@k is the correct metric. We log the per-step batch pass-rate as a coarse discovery curve.

Escape-zero (the mechanism evidence). Operational definition: a seed where **student-first stays stuck at pass = 0** (or drops back to 0) while **teacher-first escapes 0**. This is the direct signature of the flat-reward trap (§3.2): on-policy has no correct rollout to learn from, while the off-policy-leaning teacher can inject a correct solution. Replicated escape-zero is the main mechanistic contribution (§4.2).

Matched comparison. The two arms run on the *same* base and the *same* seed, so PRE matches per seed, giving a **paired (matched)** comparison that removes base- and seed-level variance. On each matched pair we count wins, ties, and losses (TF versus SF).

> **An honest note on small n.** Because the number of problems and seeds is small (n = 2–3 problems for code, n = 2 seeds for RQ1, n = 2 problems for math), the thesis reports only a **crude sign test** (for example 0.5^k over the non-tied pairs) as *descriptive, directional* evidence, never as an inferential proof. The language stays at "preliminary / directional / weak dominance"; we do not write "we prove." Every p-value is paired with a caveat about n (Chapter 4, Chapter 6).

3.7 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [3] Shenfeld et al. 2026 · [4] lasgroup SDPO repo · [5] TRL v1.4.0 docs · [6] LiveCodeBench · [7] Qwen3 · [8] GRPO / DeepSeekMath · [9] Chen et al. 2021 (pass@k) · [10] MathArena AIME 2026.

Chapter 4

Chapter 4 — Experiments

Best-case / projected draft. Numbers marked are projections extrapolated from the current real results under the stated hypotheses; they have not been run. The real, run numbers (idx39/12/64/77, the math pilot) are reproduced unmarked. See `BESTCASE_NOTES.md`.

4.1 4.0 Method extensions for the full study

Four changes turn the current pilot into the full study. (i) **Fallback removed:** the keep-best-correct fallback is dropped, so the filter never distils a copy; on a step where every teacher trajectory is judged a copy, the good pool is empty and that step distils nothing. (ii) **Medium scale:** the matched comparison is run over roughly sixteen frontier problems at six seeds, enough for a Wilcoxon signed-rank test rather than a crude sign test. (iii) **Thinking ON for code:** the code arm is run with reasoning traces, so epistemic verbalization can be measured (RQ2). (iv) **Method-bearing references and compute logging:** MATH-500 worked solutions are injected as the math privileged context, the same reasoner (Qwen3-4B) is used on both domains, and total generations plus token and GPU-time are logged for the compute-to-correct analysis.

4.2 4.1 Setup

Qwen3-4B with LoRA, on LiveCodeBench v6 (code) and AIME 2026 / MATH-500 (math). Code arm thinking ON. Per problem: reset to base, K = 15 TTT steps, evaluate the student PRE and POST. Sixteen frontier code problems (selected by model pass-rate) at six seeds. Full hyperparameters in Appendix A.

4.3 4.2 RQ-method: teacher-first vs student-first

On the medium matched set, teacher-first is at least as good as student-first on the large majority of seed-problem pairs and never materially worse, with the strongest margins on the hardest problems. A Wilcoxon signed-rank test over the paired POST pass-rates gives $\mathbf{p < 0.01}$ in favor of teacher-first, upgrading the current draft's crude sign test to a properly powered result. The escape-zero pattern — student-first stuck at zero while teacher-first lifts off — is **observed on a clear majority of the hard problems**, no longer a handful of anecdotes, and (with the fallback removed) is not attributable to any filter artifact.

The four real problems from the current draft anchor the projection: idx64 (TF 0.42 vs SF 0.05) and idx39 (TF 0.17 vs SF 0.09) are exactly the hard, escape-prone cases the medium set is built around.

4.4 4.2.5 Compute-matched comparison and compute-to-correct (RQ3)

Holding the generation budget equal (student-first at ten generations per step, matching teacher-first), teacher-first still wins: it stays at or above student-first on every problem and retains a large lead on the escape-zero cases. On the compute-to-correct frontier — discovery probability against total generations (and against wall-clock GPU-time) — teacher-first reaches a given discovery level with **roughly 2× fewer total generations** than best-of-k and matched student-first. This closes the main objection to the current draft: the advantage is not a sampling-volume artifact but a property of *which* trajectory is distilled.

problem	SF g=10	TF g=10 (real anchor)	gap
idx39 (hard)	0.13	0.17	TF ahead
idx64 (escape-zero)	0.11	0.42	TF far ahead

4.5 4.3 RQ1: the reprompt template

With six seeds the template effect is no longer borderline: on the hard problems the ordering T5 (reasoning-inducing) > T2 (anchor) > T1 (minimal) is **stable and significant**, while on easy problems the templates saturate. The template × teacher-first interaction, untested in the current draft, is run: the reasoning-inducing template helps most when paired with teacher-first on the hardest problems.

4.6 4.4 Robustness

The judge-invariance (difflib vs LLM) and leak-null (good_only vs good_bad) ablations from the current draft hold at the larger scale. With the fallback removed, the leak filter is now exact: every distilled trajectory is a genuine, independent solution, so the anti-leak claim no longer carries the "fallback may retain a copy" caveat.

4.7 4.5 RQ2: epistemic verbalization on code (thinking ON)

This is new relative to the current draft, which had no code reasoning traces. Measuring uncertainty markers (e.g. "wait", "let me reconsider", hedging) over the TTT steps, distillation from feedback-conditioned context **reduces the rate of epistemic markers**

as training proceeds, and the reduction **accumulates across steps**, matching the suppression Kim et al. [2] tie to high $I(y; c \mid x)$. Crucially, on code — where the reference is the model's own correct program rather than an answer — the suppression coincides with *improved* correctness, so here suppression reflects genuine consolidation rather than the answer-copying seen in the math leak regime. The contrast between the two regimes is the empirical core of RQ2.

4.8 4.6 Value-versus-procedure, validated (MATH-500 method-reference)

The current draft's central claim was a hypothesis: escape needs a method-bearing reference, not just a value. The full study tests it directly. Using the *same reasoner* (Qwen3-4B) on both domains removes the model confound, and MATH-500 supplies worked solutions as the privileged context. The projection: with a method-bearing reference the model **escapes on a substantial fraction of the reachable MATH-500 problems**, whereas with an answer-only reference (as in the AIME pilot) it does not. Holding the model fixed and varying only the reference content isolates the cause and **validates the value-versus-procedure boundary**.

math condition	reference	escape?
AIME, answer-only (real pilot)	the number	no (0/2)
MATH-500, worked solution	a method	yes, on reachable problems

4.9 4.7 Math pilot, fallback-free

Re-running the AIME pilot with the fallback removed sharpens the boundary: on the beyond-capability problems the teacher produces only copies, the judge correctly rejects all of them, and the student therefore receives no learning signal and stays at zero — a clean demonstration that the filter behaves as intended and the failure is purely capability- and reference-bound, not an artifact of the pipeline.

Chapter 5

Chapter 5 — Discussion

Best-case / projected draft. The interpretation below assumes the Chapter 4 projections () land in the hypothesized direction. The closing section states what changes if they do not.

5.1 5.1 Why teacher-first wins, now compute-fair

The mechanism is unchanged from the current draft: on hard problems the bare student rarely rolls a correct attempt, so student-first has nothing correct to distil, while teacher-first injects a correct trajectory generated under feedback. What the full study adds is that the advantage **survives at matched compute** (§4.2.5): at ten generations per step student-first improves but still loses, and the compute-to-correct frontier shows teacher-first reaching discovery with fewer total generations. The current draft could only say "outcome win, not yet compute win"; the best case turns this into a compute-fair claim, which is the single most important upgrade for credibility.

5.2 5.2 The value-versus-procedure boundary, validated

With the model held fixed and only the reference content varied (MATH-500 worked solution vs AIME answer-only, §4.6), escape appears exactly when the reference encodes an in-reach *method* and not when it encodes only a *value*. This converts the thesis's central idea from a characterization into a tested claim. In code the output is itself a method (a program that generalizes), so distillation installs a procedure; in answer-only math the output is a value that does not generalize, so the teacher can only copy. Supplying the worked solution gives math the missing method, and the boundary moves as predicted.

5.3 5.3 Epistemic suppression, and what it means here

The thinking-on code result (§4.5) lets us connect to Kim et al. [2]: distillation from feedback-conditioned context suppresses epistemic markers, and the effect accumulates across steps. The nuance the full study can finally state is regime-dependent: on code, where the reference is a correct program, the suppression coincides with genuine improvement (consolidation); in the math leak regime, the same surface suppression coincides with answer-copying and no improvement. Suppression of uncertainty markers is therefore not intrinsically harmful — its meaning depends on whether the reference carries a method.

5.4 5.4 A cleaner filter

Removing the keep-best-correct fallback means the anti-leak filter is now exact: every distilled trajectory is a genuine, independent solution. This removes the one place in the current draft where the judge could be silently overridden, and makes the math boundary argument airtight — the failure on beyond-capability problems is now demonstrably reference- and capability-bound, not a pipeline artifact.

5.5 5.5 Position relative to the parent paper

The contribution sharpens: a teacher-first organization of execution-guided self-distillation that accelerates test-time discovery, with a compute-fair advantage, a measured epistemic effect on code, and a validated boundary on *when* the approach helps — the question the parent paper [1] leaves open. This is the form in which the work is suitable for a conference submission.

5.6 5.6 If the projections do not hold

Honesty requires stating the downside, because each result could go the other way. If medium-n shrinks the effect, the claim reverts to weak dominance with a real but small margin. If student-first at matched compute closes the gap, the contribution becomes "teacher-first is more sample-efficient" rather than "better" — still publishable, but a different headline. If epistemic suppression does not appear on code, RQ2 stays a math-only observation. If MATH-500 with worked solutions still does not escape, the boundary is about reachable capability more than reference content, and the value-versus-procedure framing weakens to a capability story. The experiments decide; this draft shows the upside, and the real runs will report whichever way they fall.

Chapter 6

Chapter 6 — Limitations and Publication Roadmap

Best-case / projected draft. This chapter shows how the limitations of the current real draft are resolved, and what remains.

6.1 6.1 Limitations resolved relative to the current draft

The full study addresses the main limitations of the current real draft:

- **Statistical power.** Medium scale ($\approx$16 problems $\times$ 6 seeds) and a Wilcoxon signed-rank test replace the crude 0.5^k sign test, so the main effect is reported with controlled error rather than as a directional signal (§4.2).
- **Compute matching.** A matched-budget comparison and a compute-to-correct frontier replace the outcome-only result, so the advantage is shown to be compute-fair, not a sampling-volume artifact (§4.2.5).
- **RQ2 thinness.** Running the code arm with thinking ON produces reasoning traces on the domain where the method works, turning the epistemic question from a confounded math-only pilot into a measured result (§4.5).
- **The fallback.** Removing the keep-best-correct fallback makes the leak filter exact; the judge is never silently overridden (§4.4, §4.7).
- **The model and reference confounds.** Using the same reasoner on both domains and supplying MATH-500 worked solutions isolates the cause of the math non-escape and validates the value-versus-procedure boundary (§4.6).

6.2 6.2 Limitations that remain even in the best case

Honesty requires naming what more compute and time do *not* fix:

- **One model family, one scale.** All results are on a 4B-scale Qwen model at a personal compute scale. The findings are an empirical characterization in one setting, not a universal law, and not an industry-scale evaluation.

- **Effect size under scale.** A stronger model (8B and up) is expected to escape more on its own, so the teacher-first margin should *narrow*; the effect is expected to be most pronounced on models weak enough to stall unaided. Best case means the effect is well-measured and compute-fair, not necessarily large.
- **Reward shape vs reference content.** Code is both graded and method-bearing while answer-only math is both binary and value-only; even with MATH-500 these two factors co-vary unless a graded math reward (or an answer-only code task) is constructed.
- **Generalization of the epistemic result.** The suppression measurement is on one model with one marker lexicon; its generality across models and reasoning styles is untested.

6.3 6.3 Publication roadmap

The work is positioned for a conference submission a few months after graduation. The path: (1) run the medium-scale matched comparison and the compute-to-correct frontier (the headline result); (2) run the thinking-on code arm for the epistemic-suppression measurement; (3) run MATH-500 with worked-solution references and the same reasoner to validate the boundary; (4) scale to 8B to characterize how the margin changes with model strength; (5) extend to a second code benchmark to test cross-benchmark robustness. Steps 1–3 are the minimum for a paper; steps 4–5 strengthen it. Each is sized to a personal compute budget rather than an industry one.

Chapter 7

Chapter 7 — Conclusion

> **Best-case / projected draft.** Conclusions stated as established here rest on the Chapter 4 projections (); the current real draft remains the factual baseline.

This thesis asked how to accelerate test-time discovery in complex code generation via execution-guided self-distillation. The method contribution is a teacher-first organization: the self-teacher generates trajectories under execution feedback, a verifier and a judge filter them for correctness and independence (with the fallback removed, so a copy is never distilled), and the student is distilled toward those filtered trajectories, between on-policy SDPO and off-policy SFT.

In the full study the empirical picture is strong and, importantly, compute-fair. On a medium matched evaluation teacher-first significantly outperforms student-first and escapes the flat-reward trap on the hardest problems; at a matched generation budget, and on a compute-to-correct frontier, the advantage holds, so it is not a sampling-volume artifact. The reprompt-template formulation of the feedback affects discovery, most clearly on hard problems. With reasoning enabled, distillation from feedback-conditioned context measurably suppresses epistemic verbalization on code, and the meaning of that suppression turns out to be regime-dependent. Most decisively, holding the model fixed and supplying a method-bearing reference lets the model escape on math where an answer-only reference does not, validating the central value-versus-procedure boundary: escape requires the reference to encode an in-reach method, not merely a value.

Even in this best case the work is an empirical characterization in one model family at a personal compute scale, and a stronger model is expected to narrow the margin. But with the projected results in hand, the contribution is a compute-fair, mechanistically explained, and boundary-validated account of when execution-guided self-distillation accelerates test-time discovery — the form in which it is ready for publication. The immediate next step is to run the experiments this draft projects, and to report whichever way they fall.

References

[1] J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, A. Krause, "Reinforcement Learning via Self-Distillation," arXiv:2601.20802, 2026. Code: github.com/lasgroup/SDPO. [2] Kim et al., "Why Self-Distillation Degrades: Suppression of Epistemic Verbalization from Rich Context," arXiv:2603.24472, 2026. [3] I. Shenfeld et al., "Self-Distillation Fine-Tuning (SDFT) for Continual Learning," arXiv:2601.19897, 2026. [4] lasgroup, "SDPO official codebase (verl fork)," github.com/lasgroup/SDPO, 2026. (scientific reference; KL config α=1.0, top-K=20 cho LCBv6). [5] HuggingFace, "TRL v1.4.0 — SDPO/S-DFT Trainer documentation," 2026. [6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code," arXiv:2403.07974, 2024. [7] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025. [8] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv:2402.03300, 2024. (introduces GRPO.) [9] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. (pass@k.) [10] MathArena, "AIME 2026," HuggingFace dataset `MathArena/aime_2026`, 2026. URL: https://huggingface.co/datasets/MathArena/aime_2026. [11] R. Agarwal et al., "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes," arXiv:2306.13649, 2023. (GKD; generalized JSD, cited by Hübotter for JSD stability.) [12] E. D. Dolan and J. J. Moré, "Benchmarking optimization software with performance profiles," Math. Program., vol. 91, no. 2, pp. 201–213, 2002. (discovery-time / performance-profile metric lineage, per Hübotter note 8.) [13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015. (knowledge-distillation lineage anchor.)

Appendix A

Appendices

Appendices A–F are complete. B and C are reproduced verbatim from the codebase (`07` ↪ `_discovery_curve.py`, `09_teacher_first.py`); D from the W&B `output.log` files (math pilot and a code exemplar); E/F from the W&B export and reconciliation. The one remaining optional addition is the full idx0–90 code frontier-scan table (F.2), for which only the four problems actually used are listed.

A.1 Appendix A — Hyperparameters

A.1.1 A.1 Code (LiveCodeBench v6, core)

Setting	Value
Model	Qwen3-4B, LoRA r=32, bf16, thinking OFF
Optimizer	AdamW, lr = 1e-5
TTT steps K	15
Teacher samples `teacher_n` (TF)	10
`num_generations` (SF baseline)	4
Teacher temperature	~1.0
Eval samples N_{eval}	16 (student-only, PRE and POST)
Seeds	0, 1, 2, 3 (matched PRE per seed)
Template	T2 (anchor) for the main comparison; T1/T2/T5 for RQ1
KL top-K	20
KL direction α	1.0 (reverse KL)
Importance-sampling clip	2.0
Similarity threshold (difflib judge)	~0.9
Few-shot exemplars `max_fewshot`	3
Few-shot option	good_only (main); good_bad (leak ablation)
Judge	difflib (string-sim) or groq `llama-3.3-70b` (semantic)
`reference_mode`	best_in_batch
Hardware	Colab L4 (22 GB) / A100

A.1.2 A.2 Math (AIME 2026, pilot)

Setting	Value
Model	google/gemma-4-E4B-it, LoRA all-linear, thinking ON
TTT steps K	3
Teacher samples `teacher_n`	4
Eval samples	4
`max_new_tokens`	16384 (8192 truncated `\boxed`, producing spurious zeros)
Sampling	top_p 0.95, top_k 64
`reference_mode`	ground_truth (leak regime)
Judge	`zai:glm-4.5-flash`, derive-vs-copy (`is_copy` + `reasoning_quality` 1–5)
Data	MathArena/aime_2026 (30 problems, integer answers)
Hardware	Modal A100-80GB

Shared distillation core (both domains): per-token top-K reverse KL with teacher detached (self-distillation, shared weights), token-aligned over the target trajectory (§3.3.5).

A.2 Appendix B — Reprompt templates (verbatim)

Each preset overrides the SDPO template *slots* only; the model, the feedback content (the privileged context), and the hyperparameters are held fixed for a clean ablation. The TRL 1.6.0 slots are `reprompt_template` → `{prompt}{solution}{feedback}`, `feedback_template` → `{feedback_raw}` (this `{feedback_raw}` is the privileged context), and `solution_template` → `{successful_previous_attempt}`. T2 is the TRL/paper default verbatim.

Honest scope note. The codebase (`07_discovery_curve.py`, `REPROMPT_TEMPLATES`) implements **four** presets, reproduced below: T1, T2, T5, T6. The taxonomy of §3.4 also names T3 (verbose), T4 (structured JSON), and T7 (cumulative history); these three are conceptual points in the design space and were **not** implemented. Of the four implemented, the experiments probe T1/T2/T5 (§4.3); T6 is implemented but unprobed.

```
# T1 - Minimal: teacher is told the attempt was wrong but NOT why
#   (feedback_template drops the {feedback_raw} test/error detail).
"T1_minimal": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nProvide a corrected solution
↪ .\n",
    "feedback_template": "\nYour previous attempt was incorrect.\n\n",
},
# T2 - Standard / anchor: exact TRL + paper defaults (rich feedback included).
"T2_standard": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nCorrectly solve the original
↪  question.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T5 - Reasoning-inducing: SAME rich feedback as T2, trailing instruction asks
#   for root-cause analysis first.
"T5_reasoning": {
```

```
    "reprompt_template": "{prompt}{solution}{feedback}\n\nFirst, identify the root
↪ cause of the failure, then provide a corrected solution.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T6 - First-person: SAME rich feedback, reframed as the model's own reflection.
"T6_first_person": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nI attempted this problem and
↪  my solution was incorrect. Let me reconsider and write a correct solution.\n",
    "feedback_template": "\nHere is the feedback from my unsuccessful earlier attempt
↪ :\n\n{feedback_raw}\n\n",
},
```

A.3 Appendix C — Teacher and judge prompts (verbatim)

A.3.1 C.1 Teacher prompt and few-shot block

The teacher prompt is `question + few-shot block + feedback + trailing instruction`, where the feedback and trailing instruction come from the selected reprompt preset (Appendix B). A worked instance of the assembled prompt (with the leaked math reference) is shown in Appendix D.1. The few-shot block (`09_teacher_first.py`, `_build_fewshot_block`) is assembled as follows; exemplars are stripped of their thinking traces and capped at 1500 characters:

```
Here are correct, independent example solutions:

Correct example 1:
    [good trajectory, post-think solution only, fenced as a python block]

[good_bad option only -- appended after the good examples:]
Here are INCORRECT or copied attempts to avoid:

Bad example 1 (do not imitate):
    [bad / copied trajectory, fenced as a python block]
```

Under `good_only` (Option 2) only the “Correct example” exemplars appear; under `good_bad` (Option 1) the “Bad example” block is appended, which is what re-introduces reference-like content (§3.3.4).

A.3.2 C.2 LLM judge prompt — code (groq `llama-3.3-70b`)

```
You are judging a CANDIDATE solution to a competitive-programming problem.

You are given the PROBLEM, a REFERENCE solution, and a CANDIDATE solution. The
candidate is ALREADY known to be correct (it passes the test cases), so do NOT
re-check correctness. Decide only two things:

  1. is_copy: Is the candidate essentially a copy of the reference solution --
     i.e. the same algorithm, structure and control/data flow, ignoring trivial
     renaming, comments, or formatting? true if it is essentially a copy, false
     if it is an independent solution (different approach/structure).
```

```
  2. reasoning_quality: How clear and self-contained is the candidate's logic,
     on an integer scale 1-5 (1 = obfuscated/unclear, 5 = clean, clearly reasoned).

PROBLEM: {problem}
REFERENCE SOLUTION: {reference}
CANDIDATE SOLUTION: {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

A.3.3 C.3 LLM judge prompt — math (`glm-4.5-flash`)

```
You are judging a CANDIDATE solution to a math problem.

You are given the PROBLEM, the KNOWN CORRECT ANSWER, and the CANDIDATE's full
output (which may include a <think>...</think> reasoning block followed by a final
\boxed{answer}). The candidate was ALREADY shown the correct answer as feedback,
so it is trivial for it to simply restate it. Decide only two things:

  1. is_copy: Did the candidate just COPY/state the given correct answer WITHOUT
     genuine step-by-step derivation? true if it merely asserted the answer (no
     real working, or working that does not actually lead to the answer). false if
     the candidate DERIVED the answer through genuine step-by-step reasoning.
  2. reasoning_quality: How sound and self-contained is the candidate's
     derivation, on an integer scale 1-5 (1 = no/incoherent reasoning, 5 = clear,
     correct, complete derivation).

PROBLEM: {problem}
KNOWN CORRECT ANSWER: {reference}
CANDIDATE OUTPUT (including any <think> reasoning): {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

The judge returns structured JSON (`is_copy`: boolean, `reasoning_quality`: integer). Note the code judge is explicitly told the candidate is already correct and to judge only copying and clarity, which is why with thinking OFF it functions as a semantic copy-detector rather than a reasoning scorer (§3.3.3).

A.4 Appendix D — Example trajectories (math pilot)

Excerpts are verbatim from the W&B `output.log` of each run; long completions are trimmed with `[...]`. What the logs record per run: the teacher prompt, the per-step judge verdicts, and the student PRE/POST eval completions. The teacher-generated trajectories that the judge scored are *not* stored as text (only their verdicts), so copying is evidenced here by the verdicts, not by the copied text. The math excerpts (D.1–D.4) show the form-not-substance failure; the code excerpt (D.5) shows the contrasting genuine discovery.

A.4.1 D.1 Teacher prompt with the leaked reference (idx9, run `fi5m0as1`)

```
<|turn>user
Let triangle ABC have side lengths AB = 13, BC = 14, and CA = 15. Triangle triangle A’
↪ B’C’
is obtained by rotating triangle ABC about its circumcenter so that AC is
perpendicular BC, with A’ and B not on the same side of line B’C’. Find the
integer closest to the area of hexagon AA’CC’BB’.

The following is feedback from your unsuccessful earlier attempt:

Reference: the correct final answer is 156.

Correctly solve the original question.
[...]
```

The reference is the bare answer 156; there is no worked solution to distill, only the number to copy (§3.5).

A.4.2 D.2 Judge verdict sequences

idx9 (run `fi5m0as1`), correct answer 156; batch reward 1.0 every step, so all 12 trajectories are nominally correct:

step	verdicts (is_copy / reasoning_quality)	n_good / n_bad
1	(T,1) (T,2) (F,2) (F,4)	1 / 3
2	(T,2) (T,2) (T,2) (T,3)	1 / 3
3	(T,2) (T,2) (T,1) (T,3)	1 / 3

The only genuine derivation (F,4) is at step 1; steps 2–3 are all copies, so the kept “good” comes from the fallback. idx8 (run `463y4fjr`), correct answer 29:

step	verdicts	n_good / n_bad	batch reward
1	(T,1) (T,3)	1 / 1	0.50
2	(T,3) (T,2) (T,1)	1 / 2	0.75
3	(T,4) (T,3) (T,2) (F,2)	1 / 3	1.00

Every idx8 verdict is `is_copy=true` (the lone `F` has reasoning_quality $2 < 3$), so every kept “good” is a fallback-retained copy.

A.4.3 D.3 Form changes, substance does not (idx9 student eval, PRE vs POST)

PRE (23,920 chars, boxed **148**). The model recognizes the configuration is contradictory, then satisfices toward a clean angle:

> “But $AC \perp BC$ is impossible in $\triangle ABC$ [. . .] **Crucial Insight Check:** [. . .] This seems overly complex for a calculation intended to yield a clean integer answer nearby. Let’s pivot and assume θ is a ‘nice’ angle, like 90° or 180° [. . .]”

It computes 315.5 under one reading, reinterprets the hexagon as a union (147.5), and boxes 148.

POST (11,951 chars, about half the length, boxed **168**). After TTT the setup is better — it now derives $\cos C = 3/5$ and $\theta = 90° \pm C$ — but it gives up earlier and shortcuts to twice the triangle area:

> "[...] unless the overlap is significant [...] the area is often closely approximated by the sum of the individual areas [...] the most straightforward interpretation leading to a clean integer answer is that the intended area calculation ignores the slight overlap [...] Area$(H) \approx 84 + 84 = 168$."

Both are wrong (correct 156); the model never reaches or memorizes the leaked answer. TTT changed the reasoning style (more confident, more willing to shortcut), not the capability.

A.4.4 D.4 The opposite stylistic drift (idx8 student eval, PRE vs POST)

idx8 drifts the other way. **PRE** (boxed **40201**) treats the rolls as independent and computes $p = 2^{13} \cdot 3/5^6$, giving $m + n = 24576 + 15625 = 40201$. **POST** (boxed **17**) is *more* rigorous — it does casework on $R_2 = R_4$ versus $R_2 \neq R_4$, counts $N_B = 11700$ and $N_{A \cap B} = 3600$, and simplifies $p = 4/13$ to $m + n = 17$. POST is longer and more careful, yet still wrong (correct 29). Across the two problems, then, TTT moves the reasoning style in opposite directions (idx9 lazier, idx8 more rigorous) while leaving correctness unchanged.

A.4.5 D.5 Genuine discovery on code (idx12, run `teacherfirst-...-idx12`)

The contrast with the math pilot is the point of the thesis. Here the output is a *program*, and teacher-first installs a working one. Problem `abc389_b`: given X, find N with $N! = X$.

PRE (greedy score 0.075, pass@16 = 0.000). The student inverts the task, computing a factorial instead of finding N, which also overflows the integer-string limit on large inputs:

```
import math
X = int(input())
print(math.factorial(X))
```

Execution feedback (the privileged context — this is what "execution-guided" means):

```
Your solution must satisfy these test cases:
- Input: 6  ->  Expected: 3
- Input: 2432902008176640000  ->  Expected: 20
Result of running your earlier attempt:
Runtime Error
ValueError: Exceeds the limit (4300 digits) for integer string conversion ...
Last Executed Input: 5040
```

POST (greedy score 1.000, pass@16 = 1.000). The student now iterates to find N, a correct algorithm that generalizes to any input:

```
import sys
def find_n_factorial(X):
```

```
    n = 1
    current = 1
    while current < X:
        n += 1
        current *= n
    return n
def main():
    X = int(sys.stdin.readline())
    print(find_n_factorial(X))
if __name__ == "__main__":
    main()
```

PRE pass 0.000 → POST pass 1.000 (discovery curve 0.85 → 1.0 → … → 1.0; greedy 0.075 → 1.0). Unlike the math pilot, the distilled output is a method: a program that solves the problem for every input, not a value to copy. This is the value-versus-procedure boundary (Chapter 5) made concrete.

A.5 Appendix E — Per-seed results

A.5.1 E.1 idx39 (abc393_d, hard, base pass ≈ 0.1), eval-16, difflib judge

seed	PRE	TF POST	SF POST
0	0.000	0.438	0.188
1	0.000	0.062	0.000
2	0.062	0.125	0.125
3	0.188	0.062	0.062
mean		0.172	0.094

A.5.2 E.2 idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16

seed	PRE	TF POST	SF POST
0	0.000	1.000	0.938
1	0.062	1.000	0.500
2	0.062	1.000	1.000
3	0.125	1.000	0.938
mean		1.000	0.844

A.5.3 E.3 Hard-frontier means and escape-zero instances

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx39	abc393_d	difflib	0.094	0.172	2/2/0	s1 (SF 0→0, TF 0.062)
idx64	abc397_e	LLM-groq	0.047	0.422	4/0/0	s1 (SF 0.062→0, TF 0.375), s2 (SF 0→0, TF 0.062)
idx77	abc399_f	difflib	0.203	0.344	3/1/0	—

Per-seed POST pass@16 for idx64 and idx77, recovered from the W&B export and reconciled to the published means (canonical runs: idx64 = LLM judge, idx77 = difflib, both good_only / T2; dedup to the latest run per seed; `data/reconcile.py`):

seed	idx64 TF	idx64 SF	idx77 TF	idx77 SF
0	0.5625	0.000	0.250	0.0625
1	0.375	0.0625	0.3125	0.250
2	0.0625	0.000	0.6875	0.375
3	0.625	0.125	0.125	0.125
mean	0.406	0.047	0.344	0.203

The idx64 TF mean from these canonical runs is 0.406 versus the 0.422 reported in §4.2.3; the difference is which re-run is selected. Note also that the canonical idx64 seed-1 SF run here is 0.0625, whereas the escape-zero instance in §4.2.3 (SF → 0) is a different run of the same seed: the stuck-at-zero behavior is **run-dependent** (§6.1.1).

A.5.4 E.4 RQ1 template (SF arm, 2 seeds), POST pass@16

Template	idx12 (easy)	idx39 (hard)
T1_minimal	0.66	0.03
T2_standard	0.97	0.06
T5_reasoning	0.97	0.13

T5 on idx39 is stable across its two seeds (.125 / .125).

A.5.5 E.5 Judge × few-shot ablation, mean POST pass (4 seeds)

arm	idx39	idx12
SF baseline	0.094	0.844
TF · good_only · difflib	0.172	1.000
TF · good_only · LLM	0.266	0.984

arm	idx39	idx12
TF · good_bad · LLM	0.250	1.000

A.6 Appendix F — Frontier scans

A.6.1 F.1 Math (AIME 2026, Gemma-4-E4B thinking ON, n_samples = 2)

Band	Problem indices
Frontier ($0 < \text{pass} < 1$)	8, 11, 21, 25
Too-hard (pass = 0)	2, 3, 9, 10, 12, 13, 14, 16, 17, 22, 24, 26, 27, 28, 29
Ceiling (pass = 1)	0, 1, 4, 5, 6, 7, 15, 18, 19, 20, 23

The two pilot problems (idx9, idx8) are drawn from the too-hard band; idx8 was labeled frontier by the noisy 2-sample scan but is actually pass = 0 at 4 samples.

A.6.2 F.2 Code (LiveCodeBench v6, Qwen3-4B)

Problems for the matched comparisons were selected by *model* pass-rate $\in (0, 1)$ over a scan of idx0–90. The problems used are idx12 (abc389_b), idx39 (abc393_d), idx64 (abc397_e), and idx77 (abc399_f).

> **To be completed:** the full idx0–90 model-pass-rate scan table is in the logs and should be reproduced here for completeness.